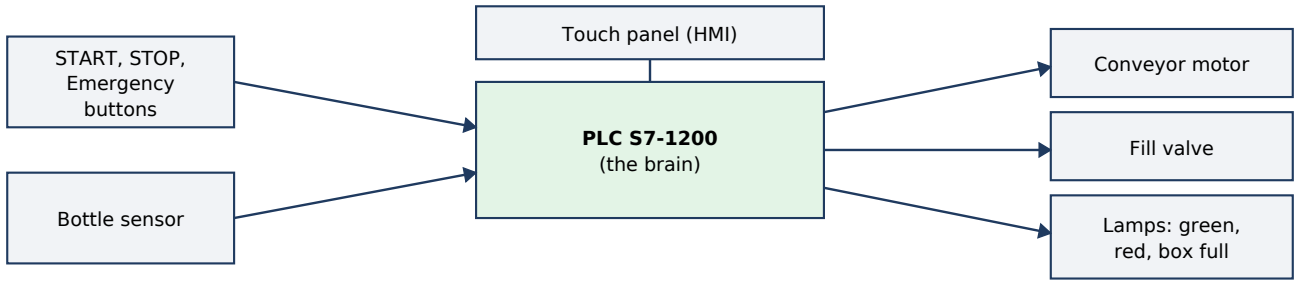
Learn PLC and HMI with One Small Machine

A friendly, compact course for TIA Portal V21: S7-1200 CPU 1214C + KTP700 Basic. You build a tiny bottle filler and test it on the laptop, no hardware needed. Version 1, 17 October 2026.

How to read this. Read it like a friend is sitting next to you. Each part is short: first the idea in plain words, then exactly what to click, then "what you just learned". Build it in a **new** project (never in the conveyor project). At the end, a table shows where the same idea is used in your real conveyor.

1. The story: what we build

Imagine a small table conveyor carrying empty bottles. A sensor sees each bottle. When a bottle arrives, the conveyor stops, a valve fills the bottle for a few seconds, then the conveyor runs again. After 6 bottles the box is full: the machine stops and a lamp blinks until someone presses RESET. There is a START and a STOP button, an Emergency button, three lamps and a touch panel.



Every part of this machine is also in your conveyor: buttons, a sensor, a motor, lamps, an alarm, an HMI. So when you understand this one, the conveyor program will look easy.

2. A PLC in one minute

A PLC is a small computer that does the same three things again and again, about a thousand times per second:

1. **Look**: read all inputs (buttons, sensor) and remember them as 0 or 1.
2. **Think**: run your program from top to bottom, using those 0s and 1s.
3. **Act**: switch the outputs (motor, valve, lamps) as the program decided.

One round is a **scan**. Because it repeats so fast, it looks like the PLC reacts at once. Two rules come from this: write each output in **one place only** (the last writer wins), and remember that a push button is 1 only **while** the finger is on it.

3. The words you need

Word	Plain meaning	In our machine
Input (I)	A wire coming IN to the PLC	%I0.0 START button
Output (Q)	A wire going OUT of the PLC	%Q0.0 conveyor motor
Memory bit (M)	A switch inside the PLC, no wire. Used to remember a decision	%M10.0 RUNNING
Tag	A name for an address, so you write START_PB instead of %I0.0	All of section 5
Bool / Int / DInt / Time	Data types: 0-1 / whole number / big whole number / a time like T#3S	FillTime_s is a DInt
Data block (DB)	A table of values inside the PLC, like a notebook page	DB_Filler
OB1 (Main)	The block the PLC runs in every scan. Everything starts here	Calls FC_Filler
FC	A block of program you write and call from OB1	FC_Filler
Network	One drawing (rung) in a block. One network answers one question	"Should the motor run?"
HMI tag	The panel's window into a PLC value	Bottle count on the screen

4. Step 1: new project, CPU, simulator

1. TIA Portal V21 > **Create new project**, name **Learn_Filler** > Create > **Project view** (bottom left).
2. **Add new device** > Controllers > SIMATIC S7-1200 > CPU > **CPU 1214C DC/DC/DC** > 6ES7 214-1AG40-0XB0. OK.
3. Double-click the CPU picture. Properties > **System and clock memory**: tick **system memory byte** (address 1) and **clock memory byte** (address 0). You now get ready-made tags: **AlwaysTRUE** (always 1) and **Clock_1Hz** (blinks once per second).
4. Save (Ctrl + S).

What you just learned: every project starts with the exact CPU. System and clock memory give free helper bits.

5. Step 2: the tags (names for every signal)

PLC_1 > PLC tags > **Default tag table**. Type each row: name, data type, address.

Name	Type	Address	What it is	Real button
START_PB	Bool	%I0.0	Green START	NO contact: 1 while pressed
STOP_PB	Bool	%I0.1	Red STOP	**NC contact: 1 while NOT pressed**
EMERGENCY_PB	Bool	%I0.2	Emergency	**NC contact: 1 while healthy**
BOTTLE_SENSOR	Bool	%I0.3	Sensor sees a bottle	1 = bottle there
CONVEYOR_MOTOR	Bool	%Q0.0	Motor runs	
FILL_VALVE	Bool	%Q0.1	Valve open	
LAMP_GREEN	Bool	%Q0.2	Running lamp	
LAMP_RED	Bool	%Q0.3	Alarm lamp	
LAMP_BOX_FULL	Bool	%Q0.4	Box-full lamp (blinks)	
RUNNING	Bool	%M10.0	Machine started (remembered)	

Name	Type	Address	What it is	Real button
FILLING	Bool	%M10.1	A bottle is being filled	
BOX_FULL	Bool	%M10.2	6 bottles done	
ALARM_EMERGENCY	Bool	%M20.0	Emergency happened (stays until reset)	
ALARM_WORD	Word	%MW20	M20.0 - M21.7 as one word, for the HMI alarm	

Why STOP and Emergency are NC. If a wire breaks, an NC input becomes 0, which looks like "STOP pressed": the machine stops safely. That is why, on your real panel, I0.3 and I0.4 must be **ON** when nobody touches them (and why everything failed when 1M was not connected to 0 V).

6. Step 3: the data block (the notebook)

Program blocks > **Add new block** > Data block > name **DB_Filler** > OK. Add these rows. In the column **Start value** type the numbers shown.

Name	Type	Start value	Used for
HMI_Start	Bool		START button on the panel
HMI_Stop	Bool		STOP button on the panel
HMI_Reset	Bool		RESET button on the panel
FillTime_s	DInt	3	Fill time in seconds, typed on the panel
FillTime	Time		The same time in the format a timer needs
BoxSize	Int	6	Bottles per box, typed on the panel
BottleCount	Int		Bottles filled so far, shown on the panel
SensorMem	Bool		Memory for the edge detection (network 4)

What you just learned: a DB keeps values that belong together. The HMI can read and write them. (This DB stays **optimized**, the TIA default: we use names only. Your conveyor turns optimization off because it uses fixed addresses like DB2.DBW0.)

7. Step 4: the program, one network at a time

Program blocks > Add new block > **Function (FC)**, name **FC_Filler**, language **LAD**. Build the networks below in this order. For each: first read the sentence, then draw it. Contacts in a row = **AND**, one below the other = **OR**, the slashed contact = **NOT**.

Network 1: "Was Emergency pressed?" (set coil)



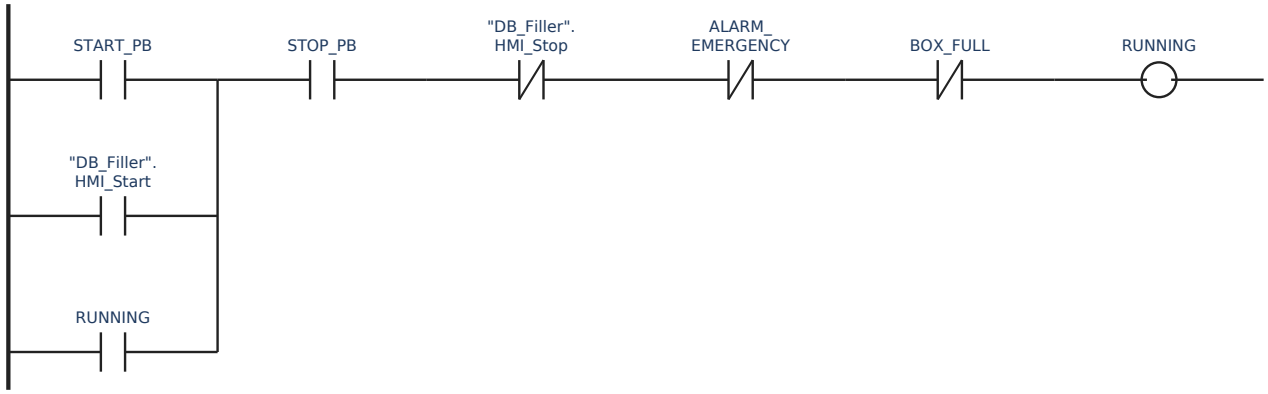
The input is 0 when Emergency is pressed, and the slashed contact passes on 0. **Set coil (S)**: once set, it stays 1 even after the button is released. That is how an alarm "remembers".

Network 2: "Clear the alarm" (reset coil)



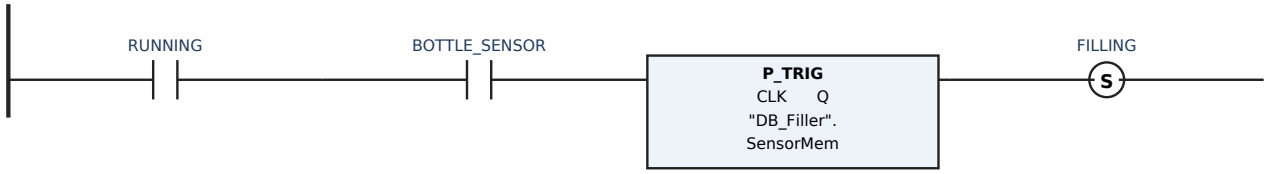
Reset works only if Emergency is released again (input back to 1). **Reset coil (R)** writes 0.

Network 3: "Is the machine running?" (the latch)



Read it: (START **or** panel START **or** already RUNNING) **and** STOP not pressed **and** no panel STOP **and** no alarm **and** box not full. The RUNNING contact under START is the **latch**: after you release START, RUNNING keeps itself on. Any condition on the right breaks it. To draw the OR: select the wire before START, click **Open branch**, add the contact, drag the branch end back to the main line.

Network 4: "Did a new bottle arrive?" (edge detection)



The sensor stays 1 while the bottle stands there. **P_TRIG** gives 1 for **one scan only**, at the moment the sensor changes from 0 to 1, so each bottle counts once. It is in Basic instructions > Bit logic. Type the memory bit under the box.

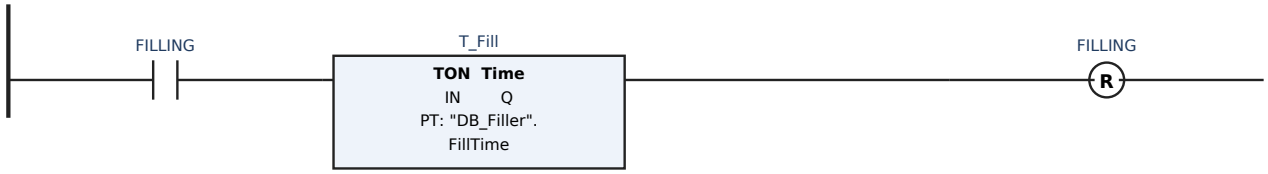
Network 5: "Convert seconds to timer time" (an SCL line)

Right-click the network title > **Insert SCL network**, and type exactly:

```
"DB_Filler".FillTime := DINT_TO_TIME("DB_Filler".FillTime_s * 1000);
```

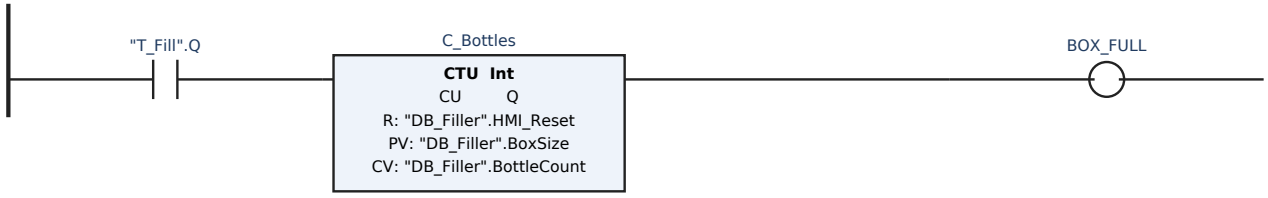
A timer counts milliseconds. 3 seconds x 1000 = 3000 ms. DINT_TO_TIME changes the number into the Time type. SCL is text programming: some things (maths) are shorter in text than in ladder.

Network 6: "Fill for a few seconds" (on-delay timer)



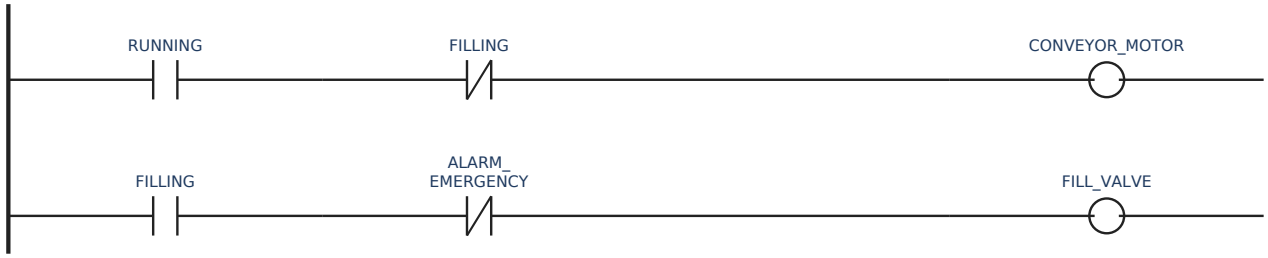
Drag **TON** from Basic instructions > Timer operations. When it asks "Call options", choose **Single instance** and name it **T_Fill**: TIA makes a small DB for the timer. While FILLING is 1, the timer counts. After PT, Q becomes 1 and resets FILLING: the conveyor runs again.

Network 7: "Count the bottles" (counter)



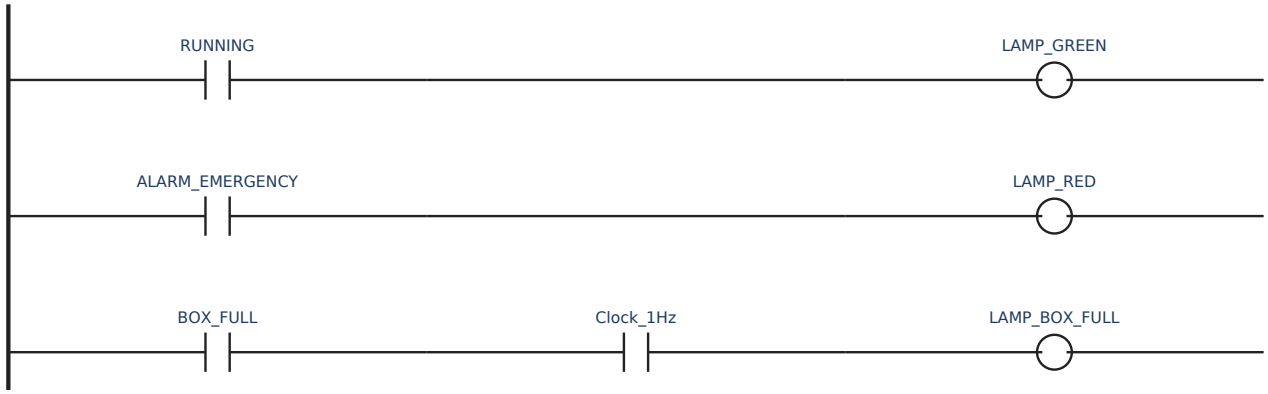
CTU (count up) adds 1 each time CU goes from 0 to 1, so once per finished bottle. When the count reaches PV (6), Q = 1 = BOX_FULL. R (reset) sets the count back to 0. Single instance, name **C_Bottles**.

Network 8: "Drive the motor and the valve" (outputs in one place)



The motor runs while the machine is running and no bottle is being filled. The valve opens while filling, but never during an alarm (safety first).

Network 9: "Which lamps are on?" (clock memory blink)



Clock_1Hz changes every half second by itself, so in series with BOX_FULL the lamp **blinks**.

Call it from OB1 and compile

1. Open **Main [OB1]**. Drag **FC_Filler** from the project tree onto network 1.
2. Click **PLC_1** > **Compile**. The Info tab must say **Errors: 0**. A red line? Double-click it: usually a typing mistake in a name.

Summary: the instructions you used

You used	Instruction
Contact NO / NC	-| |- -|/|-
Coil, set coil, reset coil	-( )- -(S)- -(R)-
AND / OR / latch	series / branch / self-holding
Edge detection	P_TRIG
Timer	TON
Counter	CTU
Maths and type change	SCL: *, DINT_TO_TIME
Blink	Clock_1Hz

8. Step 5: the touch panel (HMI)

1. **Add new device** > HMI > SIMATIC Basic Panel > 7" Display > **KTP700 Basic** 6AV2 123-2GB03-0AX0. In the wizard: PLC connections > Browse > **PLC_1**. Untick the extra screens. Finish.
2. Open the screen the wizard made (Screens > **Root screen**). Drag objects from the **Toolbox** on the right, as in the table.

3 For every object, the tag is picked with the "..." button: you can pick PLC tags and DB rows directly; TIA creates the HMI tag for you.

Object	Text / settings	Tag	Event or animation
Button	START	DB_Filler.HMI_Start	Events > **Press** > SetBitWhileKeyPressed
Button	STOP	DB_Filler.HMI_Stop	Press > SetBitWhileKeyPressed
Button	RESET	DB_Filler.HMI_Reset	Press > SetBitWhileKeyPressed
I/O field	Mode **Input**, format 99	DB_Filler.FillTime_s	The operator types the fill time
I/O field	Mode **Input**, format 99	DB_Filler.BoxSize	Bottles per box
I/O field	Mode **Output**, format 99	DB_Filler.BottleCount	Shows the count
Circle	"Motor" text next to it	CONVEYOR_MOTOR	Animations > Appearance: 0 = grey, 1 = green
Circle	"Valve"	FILL_VALVE	Appearance: 0 = grey, 1 = blue
Circle	"Box full"	BOX_FULL	Appearance: 1 = yellow, flashing
Alarm view	Toolbox > Controls	-	Shows the alarm texts

The alarm text

HMI_1 > **HMI alarms** > Discrete alarms: text **EMERGENCY STOP PRESSED**, trigger tag **ALARM_WORD**, trigger bit **8**.

Why bit 8 for M20.0? In a Siemens word the first byte is the high half. MW20 = byte M20 (bits 8 - 15) + byte M21 (bits 0 - 7). So M20.0 is bit 8. Your conveyor uses the same trick: its two alarms are bits 8 and 9 of DB2.DBW0.

What you just learned: buttons write bits, I/O fields read and write numbers, animations show bits, alarms watch bits of a word.

9. Step 6: test it on the laptop (no hardware)

1 Click PLC_1 > toolbar **Start simulation** (PLCSIM). Load the program into the simulated PLC, choose RUN.

2 In PLC_1 add a **Watch table** with START_PB, STOP_PB, EMERGENCY_PB, BOTTLE_SENSOR, RUNNING, FILLING and "DB_Filler".BottleCount. Switch monitoring on (glasses icon).

3 **First set the NC inputs to 1**: STOP_PB = 1 and EMERGENCY_PB = 1 (modify value, lightning icon). Otherwise the machine never starts: just like your real panel without 1M.

4 Click HMI_1 > **Start simulation**. A panel window opens on the laptop.

#	Do this	You must see
1	Press START on the simulated panel	RUNNING = 1, motor circle green
2	Set BOTTLE_SENSOR = 1	FILLING = 1, motor off, valve on
3	Wait 3 s, then set BOTTLE_SENSOR = 0	Valve off, motor on, BottleCount = 1
4	Repeat 2-3 five more times	BottleCount = 6, machine stops, box-full blinks
5	Press RESET	Count 0, box-full off; START works again
6	Set EMERGENCY_PB = 0	Alarm text, red lamp, machine stops
7	EMERGENCY_PB = 1, then RESET	Alarm clears
8	Type 5 in the fill time field	Next bottle fills 5 s

Tip: open FC_Filler and switch on **monitoring** (glasses). Green lines = signal flows. Watching the rungs change is the fastest way to learn.

10. Now look at your real conveyor

Idea in this course	Where your conveyor uses it
NC buttons are 1 at rest	I0.3 STOP and I0.4 Emergency (the 1M fault you fixed)
Set / reset alarm	FC_ALARMS networks 1 - 3
Latch (self-holding)	FC_AUTO network 1: START with ACTIVE in parallel
Outputs written in one place	FC_OUTPUTS: Q0.0 - Q0.5
Edge, timer, SCL line	The switched-off sensor cycle in FC_AUTO (P_TRIG, TON, SCL network 2)
Compare instead of a sensor	FC_AUTO network 3: run while register 40001 (DATA1) = 0
HMI buttons SetBitWhileKeyPressed	FWD JOG, REV JOG, FAULT RESET
Alarm bit 8 of a word	Alarms on bits 8 and 9 of DB2.DBW0

Three small exercises

- Add a second lamp output that is ON while the valve is open.
- Make the box-full lamp blink faster: use Clock_2Hz.
- Add a counter of all bottles ever filled (a second CTU without reset), and show it on the panel.